

Object Oriented Frontend JavaScript

I've been experimenting with ways of writing JavaScript in the frontend, mainly leaning on object-oriented patterns. I'd like to share the problems I've been trying to fix, and the solutions I've found, and see if they chime with what the rest of you have been up to.

a basic submission app

Let me tell you a story.

There's an HTML page, in the submissions app, for adding new authors to a draft submission. It's got a lot going on with it, as you can imagine, but I want us to zoom in on two parts.



Institution Editor



Author Editor

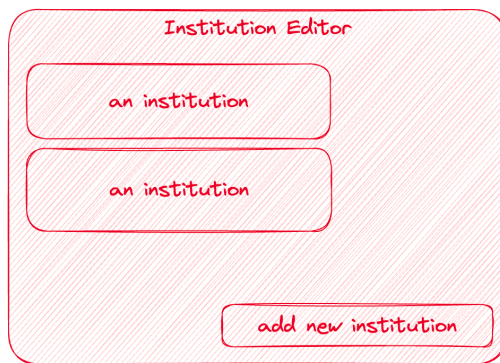
Author Editor

The first player in our drama is called the Author Editor. It's a series of HTML controls that are there to add, remove, and edit the authors of a draft submission.

So when a new author is added, it appears in a list in the Author Editor



Institution Editor

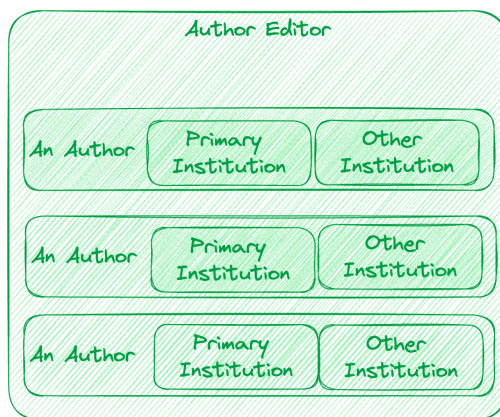


The second we'll call the Institutions Editor. This is a slightly smaller collection of HTML controls to add and remove the institutions to a draft submission.

So when a new institution is added, it appears in a list in the Institutions Editor.

So far so good.

Primary and Other Institutions



Now the minor complication is that, inside each new author that you add, there are two dropdowns that let you select the primary and other institutions of the author.

Use Cases

When I add an author

The primary institution dropdown has all the institutions

And the secondary institution dropdown does too

When I add an institution

The primary institution dropdown for each author is updated

And the secondary institution dropdown for each author is updated

So that, every time the user adds an author, we need to add all of the institutions from the institution editor to each dropdown.

And, to add to the fun, we also need to update all the inputs with a new institution when one is added in the institution editor.

So how do we approach this?

Event Listeners

```
el.addEventListener('submit', event => {
  // DO STUFF
})
```

So, whatever we do, we're going to have to listen for changes in elements in the DOM. This is achieved with our trust friend `addEventListener`: we find an element - a form, an input, anything - and we register a callback with it that triggers when a particular sort of event triggers.

So we will likely have two event listeners at work - one which triggers when someone clicks on the 'add author' button, and one that triggers when we click the 'add institution' button.

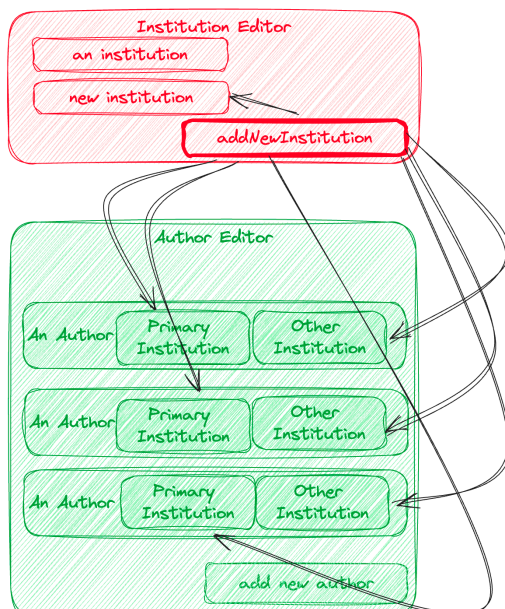
Note that - we're simplifying a bit here - there are really more than this number of event listeners at play. We'd need them on changing the institution too, for instance. But the principle remains the same.

So, what are our approaches?

Spaghetti Solution

Well, the one approach, and it's pretty much the default approach, is what I'd call the spaghetti code way. Manipulate the relevant bits of the DOM in the event listeners. It looks kinda like this...

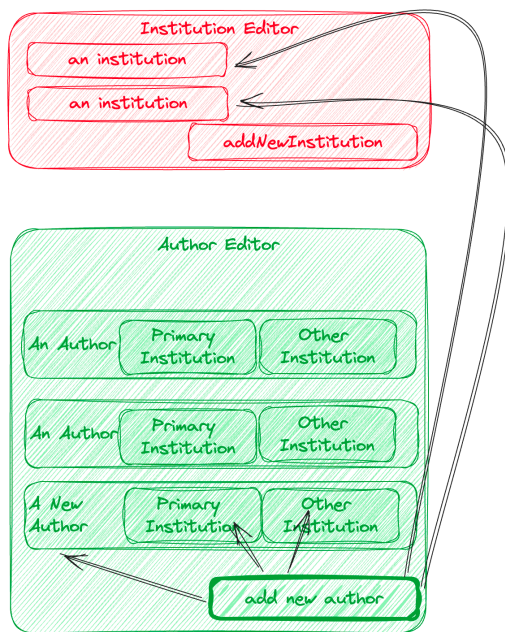
Spaghetti Adding an Institution



For adding an institution, so listening to some control in the institution editor section, we need to add the new institution element, that's fine.

But we also need to go out and find every single institution dropdown in the author editor section and then update each one.

Spaghetti Adding an Author



and the same goes for adding an author - we add the author, sure, but we then need to find out what all of the available institutions are in the institution editor section, so we reach out over into that section, bring back some data, and use that to update.

Advantages

Easy...

... at the beginning



Disadvantages

Hard to Read

Hard to Reason About

Low cohesion

High coupling

Hard to Change

It's hard to read.

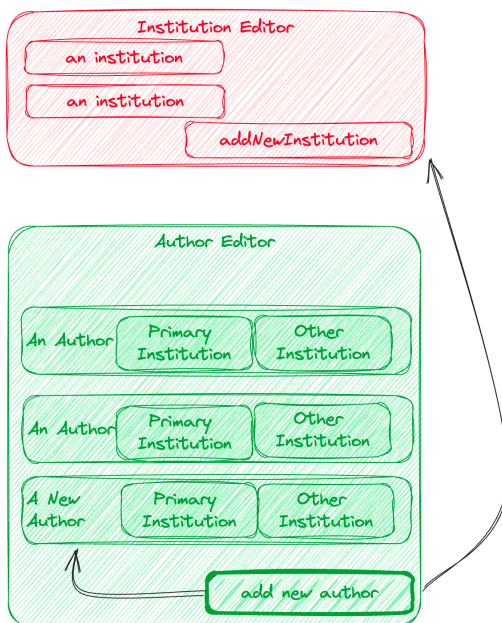
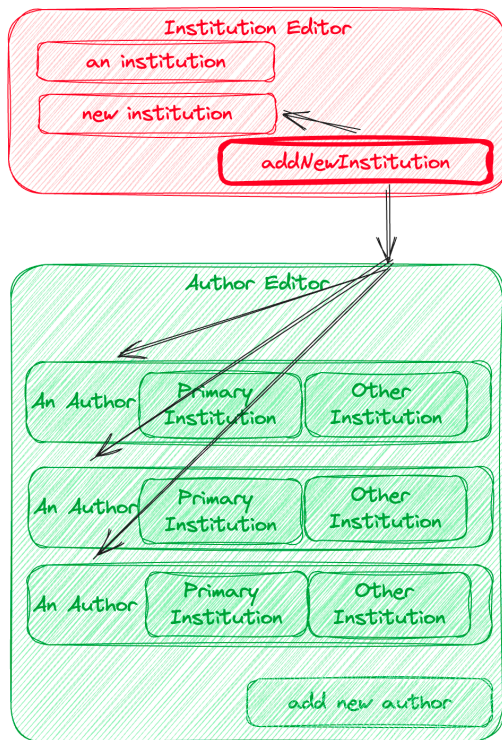
It's hard to reason about.

The bits that do things with an author are not in one place (low cohesion).

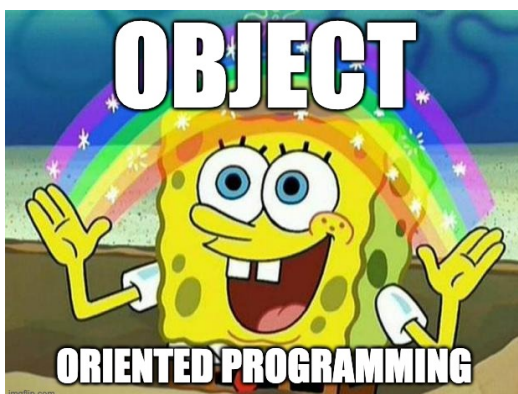
And every bit of code is intimately aware of the internal structure of the html (tightly coupled).



What we want is something more like this - where adding an institution changes one "thing" - the author editor. And the author editor takes care of the rest of the changes that need to happen - it tells the individual authors that they need to update their institutions, and then *they* change their dropdowns.



and vice versa - we don't want to dive inside the institutions 'box' to get all the institutions by searching for the right elements, we just want to ask for a list of institutions.



And the answer, at least I think, is object-oriented programming, which has a long and rich history of being used to build graphical UIs. In fact, they were pretty much invented at the same time and in the same place.

Here's how to do it.

Encapsulate!

One Component

=

One Object

HTML is your data. Well, more precisely, the DOM is your data here. So, make one object *own* one area of the DOM. This doesn't have to be neat, or perfect, but just starting by saying that an `AuthorEditor` object owns all of the data - the HTML, the DOM - for editing authors - really cleans up this model.

And what you're saying here is that this bit of the DOM, and all of its children, should only be accessed by this object.

Author Editor

```
class AuthorEditor {  
  constructor() {  
    this.authorList = document  
      .querySelector('#author-list')  
  }  
}
```

We can be a bit direct and just get hold of the elements you need in the constructor directly

Institution Editor

```
class InstitutionEditor {  
  constructor(instEdElement) {  
    this.editor = instEdElement  
  
    this.addButton.addEventListener('click', e => {  
      this.handleAddInstitution(e)  
    })  
  }  
  
  get addButton() {  
    return this.editor.querySelector('#add-inst')  
  }  
}
```

Or you can pass an 'entry point' element - the top level element of the component under which you own everything - and then get the elements you need from that.

Either way - encapsulate!

Add Messages

The effect of this is that, when you need to tell your HTML to change from the outside of your object, you need to tell the object, not the HTML.

So we write some nice declarative methods to do this

Author Editor

```
class AuthorEditor {  
  
    updateInstitutions(insts) {  
        for (author in this.authors) {  
            author.updateInstitutions(insts)  
        }  
    }  
}
```

We need to update the author institutions - so do it like this.

Institution Editor

```
class InstitutionEditor {  
  
    get institutions() {  
        return this.editor  
            .querySelector('#inst-list')  
    }  
}
```

We need to get hold of the current institutions? We do it like this.

Wire It Together

Now you just need to tell your objects about each other - we need to pass around their references so they can communicate using these nice messages we've been making.

Institution Editor

```

class InstitutionEditor {
  constructor(authorEditor) {
    this.authorEditor = authorEditor
  }

  handleAddInstitution(event) {
    // manipulate HTML
    this
      .authorEditor
      .updateInstitutions(this.getInstitutions())
  }
}

```

Author Editor

```

class AuthorEditor {
  function initialize(institutionEditor) {
    this.institutionEditor = institutionEditor
  }

  handleAddAuthor(event) {
    const insts = this.instEd.getInstitutions()

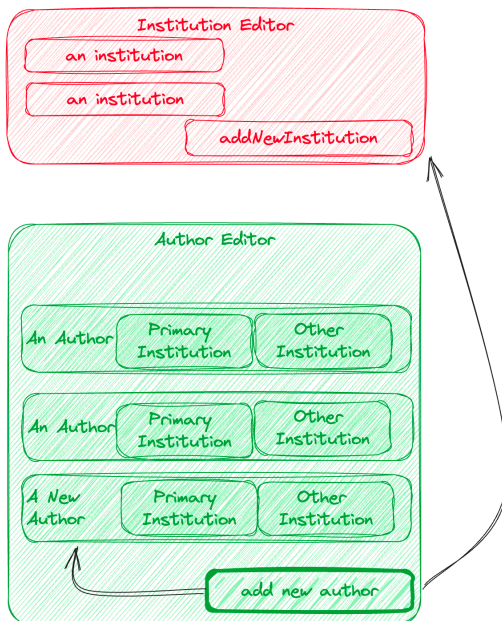
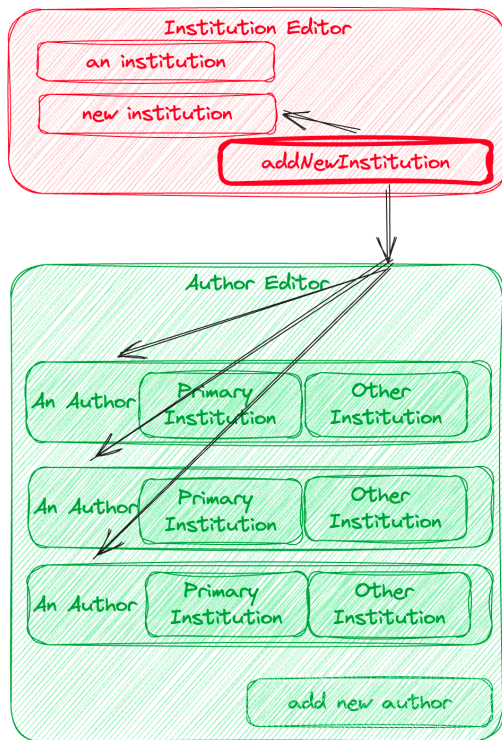
    const author = new Author(insts)
    this.addAuthor(author)
  }

  addAuthor(author) {
    // jam some HTML into the DOM
  }
}

```

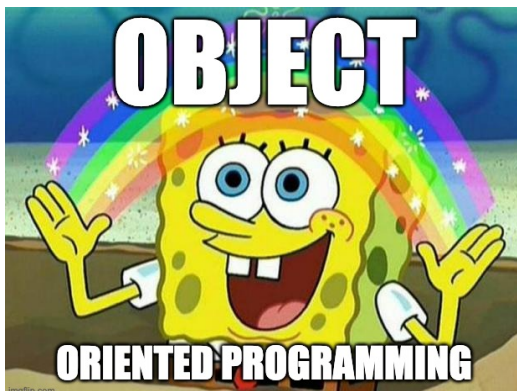
We don't need to wire together our objects in constructors - and, as you can see, this would be impossible with these two classes. We can just call initialization functions on the objects to get it the references it needs to do its job.

So the institution editor only knows about the author editor - it no longer knows anything about the internals of the DOM that the author editor wraps. It just tells the author editor that a new institution has been added... and the author editor is in charge of what changes it needs to make.



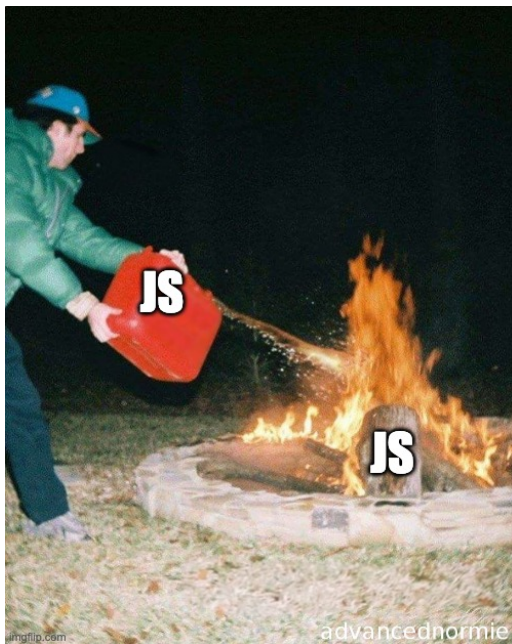
and vice versa - the author editor doesn't dive inside the list of DOM elements in the institution editor.

Cleanliness



So, that's it. Real objects, things that encapsulate their data, with short, descriptive methods.

This isn't rocket science - this is not some frontend framework magic. This is just general software engineering principles applied to the frontend.

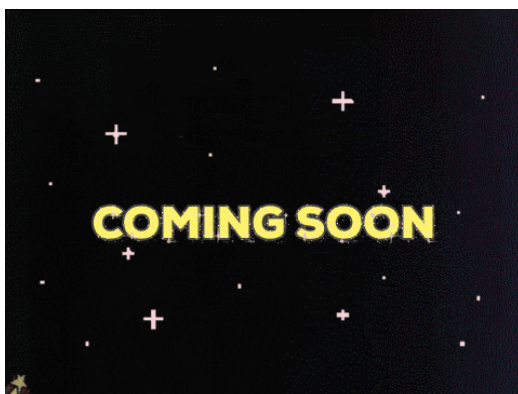


It sounds easy, and it looks easy, but too often when we're faced with "the frontend", and a wall of preexisting spaghetti code, all taste and decency goes out of the window when "backend" developers get in there, as we bang in just one extra line of code, just trying to get out of the hellscape.

It's far too easy to add to the tyre fire.



When, really, you know, it's just *code*. It's just what you *do* already. With a bit of patience and love, a bit of care and attention, with the application of simple engineering skills *that you should already use every day*, you can make frontend code in vanilla JavaScript that's a pleasure to work with.



Sometime next week I'll talk about further reducing the coupling of your components using the publish-subscribe pattern, including how to write an event bus in JavaScript in one line of code.

So yeah, that's it. Are you already doing something similar in the frontend? Are you creating the same encapsulations in a different way? Do you think this is just over engineering and we should be doing something else?

Questions?